Reviewer Pack — Invariant Space Dimension Gap in Permanent vs Determinant Te…

Entry cf52d7a98cd5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 21:42:36 UTC

Invariant Space Dimension Gap in Permanent vs Determinant Tensor Powers

Entry ID: `cf52d7a98cd5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 21:42:36 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Algebraic Groups **Field B** (complexity object): Tensor Rank of Permanent vs Determinant

Statement:

> For all n ≥ 2, the dimension of the space of GL_n-invariant polynomials on the n-th tensor power of the permanent polynomial exceeds that of the determinant polynomial by a factor of Ω(2^n).

Rationale (proposer's reasoning):

> Invariant spaces capture symmetry under linear transformations. If permanent's invariant dimension grows exponentially faster than determinant's, it suggests a structural complexity barrier aligning with GCT's orbit closure separation goals.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b387d9634450df2c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_mult(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def matrix_det(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * matrix_det(submatrix)
        return det

    def invariant_dimension(matrix, n):
        permanent_rank = 0
        determinant_rank = 0

        # Compute permanent rank
        perm_matrix = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        for _ in range(10):  # Sample multiple permutations to estimate rank
            permuted_matrix = [random.sample(row, len(row)) for row in perm_matrix]
            permanent_rank += matrix_det(permuted_matrix)

        # Compute determinant rank
        det_matrix = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        for _ in range(10):  # Sample multiple permutations to estimate rank
            permuted_matrix = [random.sample(row, len(row)) for row in det_matrix]
```

```
            determinant_rank += matrix_det(permuted_matrix)

        return permanent_rank, determinant_rank

    n_values = [5, 10, 15, 20, 30, 40]
    total_permanent_rank = 0
    total_determinant_rank = 0
    instances_tested = 0

    for n in n_values:
        matrix = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        permanent_rank, determinant_rank = invariant_dimension(matrix, n)
        total_permanent_rank += permanent_rank
        total_determinant_rank += determinant_rank
        instances_tested += 1

    if instances_tested == 0:
        return {
            "metric_name": "permanent_vs_determinant_ratio",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    ratio = total_permanent_rank / instances_tested / (total_determinant_rank / instances_tested)
    return {
        "metric_name": "permanent_vs_determinant_ratio",
        "metric_value": ratio,
        "instances_tested": instances_tested,
        "conjecture_holds": ratio > 2 ** n_values[0],
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_ratio = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}"
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        result = f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={fir
    else:
        result = "RESULT: INCONCLUSIVE reason=unknown"

    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended time limits and memory allocation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	117355
2	preregistration	ollama_remote	qwen3:8b	0	0	32817
3	novelty	ollama_remote	qwen3:8b	0	0	28792
4	novelty	ollama_remote	qwen3:8b	0	0	21424
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16504
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11236
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9249
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10687
9	judge	ollama_remote	qwen3:8b	0	0	20886

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 268950 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cf52d7a98cd5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cf52d7a98cd5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cf52d7a98cd5.tar.gz` (if generated)